TP 14 : Voyageur de commerce

Ce TP est à réaliser en C.

Préliminaires

On considère le problème `Voyageur de commerce` suivant :

Instance :	Un graphe complet G muni d'une fonction de poids p
Solution :	Un cycle hamiltonien.
Fonction à minimiser :	Le poids total du graphe

FIGURE 1 – Vive la randonnée !

L'objectif de ce TP est d'étudier un algorithme d'approximation dans le cas où la fonction de poids vérifie l'inégalité triangulaire :

$$\forall r, s, t \quad p(r,t) \leq p(r,s) + p(s,t)$$

C'est notamment le cas lorsque le graphe représente des positions dans l'espace et que le poids correspond à la distance entre les sommets.

On s'intéressera donc en réalité au sous problème suivant : `Voyageur de commerce dans le plan`

Instance :	Un ensemble de points dans le plan à deux dimensions. On munit le plan de la fonction de distance suivante : $(p_1, p_2) \mapsto d(p_1, p_2)^2$ où d est la distance euclidie
Solution :	Un cycle hamiltonien.
Fonction à minimiser :	Le poids total du graphe

Définitions

On définit le type `Coordonnees` de la façon suivante :

```
typedef struct
{
    int x;
    int y;
} Coordonnees;
```

Il s'agit d'une représentation d'un point dans le plan en deux dimensions.

On définit le type `AretePonderee` qui nous permettra de manipuler des arêtes pondérées indépendamment les unes des autres :

```
typedef struct
{
    int u;
    int v;
    int poids;
} AretePonderee;
```

Réduction

On cherche dans cette partie, à partir d'un ensemble de points du plan, à créer un graphe complet qui lui correspond.

Question 1 : Écrire une fonction `distance(Coordonnees p1, Coordonnees p2)` qui étant donnés deux points, renvoie leur distance.

Question 2 : Écrire une fonction `int** creer_graphe(Coordonnees* points, int n)` qui, étant donnée un tableau de n points, renvoie le graphe des distances de ces points sous forme de matrice d'adjacence. On les numérotera dans l'ordre qui nous arrangera.

Approximation

On cherche à construire un cycle hamiltonien de poids aussi faible que possible. Pour cela, on commence par construire un arbre couvrant de poids minimal.

Question 3 : Écrire une fonction `void extraire_aretes(int** graphe, int n, AretePonderee* aretes, int * nb_aretes)` qui, étant donné un graphe pondéré, renvoie la liste des arêtes pondérées. On retirera les boucles lors de cette étape.

Question 4 : Écrire une fonction `int** contruire_matrice(AretePonderee* aretes, int* nb_aretes, int n)` qui, étant données une liste d'arêtes d'un graphe à n sommets, renvoie une matrice d'adjacence. Celle-ci ne sera pas pondérée.

Question 5 : Écrire une fonction `void tri_aretes(AretePonderee* aretes, int nb_aretes)` qui, étant donnée une liste d'arêtes, la trie par poids croissant.

On représente maintenant la structure unir et trouver par des arbres comme vus en cours. Pour cela, on manipule deux tableaux :`parent` qui pour chaque sommet, indique son parent et `rang` qui indique la taille du sous-arbre enraciné en ce sommet.

```
typedef struct
{
    int* parent;
    int* rang;
} UnirTrouver;
```

Question 6 : Écrire une fonction `int trouver(UnirTrouver uf, int x)` qui renvoie le représentant de x dans cette représentation.

Question 7 : Écrire une fonction `void unir(UnirTrouver uf, int x, int y)` qui réalise l'union des deux composantes de x et y.

Question 8 : En utilisant l'algorithme de Kruskal, écrire une fonction `void arbre_couvrant_minimal(int** graphe, int n, AretePonderee* arbre, int* taille_arbre)` qui, étant donné un graphe, renvoie une liste d'arêtes correspondant à un arbre couvrant de poids minimal.

Question 9 : Écrire une fonction `int* parcours_arbre(int** adj, int n)` qui, étant donné un arbre et une racine, renvoie le parcours des sommets de l'arbre sous forme de liste. On pourra utiliser une fonction auxiliaire de signature `void parcours_aux(int* adj, int n, int* chemin, int* indice, bool* deja_vus)`

Question 10 : Écrire une fonction `int* eliminer_doublon(int* chemin, int n)` qui, étant donné un cycle renvoyé par la fonction précédente, renvoie le cycle hamiltonien qui en découle.

Question 11 : Écrire une fonction `int* construire_cycle(Coordonnees* points, int n)` qui, étant donnée une liste de points dans le plan, renvoie le cycle hamiltonien ainsi obtenu.